中国研究生操作系统开源创新大赛

项目创新说明

作 品 名 称：面向智能机器人的 AIOS
参 赛 队 伍：Xenoblade
完 成 日 期：2026 年 10 月

目 录

创新点 1 在 StarryOS 上构建完整的异构硬件视觉链路 1
创新点 2 提出面向控制实时性的“最新帧优先”采集机制 1
创新点 3 提出面向硬件完成事件的低时延调度方法 1
创新点 4 设计任务状态驱动的异构目标感知方法 1
创新点 5 将目标检测结果直接转化为柔性运动控制 1
小 结 2

创新点 1　在 StarryOS 上构建完整的异构硬件视觉链路

面向 SG2002 平台，本项目补齐 UVC、JPU、VPSS、ION 与 TPU 五类驱动能力，把原本压在 CPU 上的解码、 色彩转换与缩放交给片上加速器，形成一条端到端的硬件加速链路：

$$\text{MJPEG} \rightarrow \text{JPU} \rightarrow \text{YUV422P} \rightarrow \text{VPSS} \rightarrow \text{RGB Planar} \rightarrow \text{TPU}$$

JPU 与 VPSS 之间通过 ION 共享物理缓冲区传递图像，整幅 YUV 与 RGB 图像不再经 CPU 反复转换与复制。 在此基础上，StarryOS 的单帧视觉时延为 53.024 ms，在相同的软硬件条件下追平并略优于 Linux 基线（54.23 ms）。

创新点 2　提出面向控制实时性的“最新帧优先”采集机制

先进先出队列的目标是处理每一幅图像；当推理速度低于摄像头帧率时，队列中会积压历史帧，机器人实际 使用的图像越来越旧。对运动控制而言，图像是否足够新，比是否处理过每一帧更重要。本项目据此把优化 目标从“无丢帧”改为“低帧龄”：后台异步采集持续组帧，双缓冲发布保证消费者读到的始终是完整快照，帧序号同步让控制线程只取序号更新的图像，一帧背压则限制采集任务对单核 CPU 的占用。控制线程由此 始终拿到最新且完整的图像。

创新点 3　提出面向硬件完成事件的低时延调度方法

在单核 StarryOS 上，加速器已经完成、提交线程却不能及时恢复，是软件长尾的主要来源。本项目为 TPU 与 VPSS 建立分段时间戳体系，把一次设备调用拆成任务提交、硬件执行、中断到达、线程恢复与返回用户态 等阶段，使“硬件慢”与“调度慢”能够分别度量。在此基础上，通过 TPU 同步直通执行、TDMA-only 短任务 分类、中断后运行队列前插以及轮询—中断竞态修复，消除了数十毫秒的软件排队与唤醒长尾。该方法的落点 是按硬件任务类型与完成事件重新设计调度路径，单纯提高线程优先级触及不到排队与唤醒环节。

创新点 4　设计任务状态驱动的异构目标感知方法

未持球时，机器人用 YOLOv8 与 TPU 识别网球；抓到球之后切换到基于 YUV 色彩特征的红色容器检测， 并跳过这一阶段不再需要的 VPSS 与 TPU 推理：

$$\text{未持球} \Rightarrow \text{YOLO 网球检测}$$

$$\text{已持球} \Rightarrow \text{红色容器检测}$$

两种任务状态各自选用最合适的感知算法，避免用一个更大、更复杂的多类别模型覆盖全部任务，在保持 识别能力的同时减少计算量，实现感知算法与任务状态的协同优化。

创新点 5　将目标检测结果直接转化为柔性运动控制

系统用检测框中心位置估计目标方向，用检测框面积占整幅图像的比例近似目标距离，并据此设计二次减速 函数：

$$v(a) = v_{\text{chase}} \left(1 - \frac{a}{a_{\text{grab}}}\right)^2$$

目标较远时机器人保持较高速度；随着目标面积增大，车速非线性下降；进入抓取区域后停车并启动机械臂。与固定速度或线性减速相比，这一方法可以减少近距离撞球、转向过冲与左右摇摆。

小结

本项目的核心创新，是面向移动机器人实时闭环，把 StarryOS 内核驱动、异构硬件加速、最新帧调度、任务状态感知与非线性运动控制做跨层协同设计。系统在 StarryOS 上打通了 JPU—VPSS—TPU 低拷贝视觉 链路，用完成事件感知的调度与最新帧机制压缩软件长尾和图像帧龄，并把视觉结果直接用于连续抓球、放球与再次搜索，形成了从操作系统内核到机器人行为决策的完整优化闭环。